

Hierarchical K-TSP Classifier platform

1. Project Contributions

This project develops and evaluates a **Hierarchical K-Top Scoring Pairs (K-TSP) classifier** for cancer-related gene expression data.

The original objective was to reproduce and extend a K-TSP-based classification pipeline and investigate how preprocessing, feature selection, model configuration, and computational architecture affect classification performance and scalability.

My contribution focused not only on implementing the classifier, but also on understanding **which parts of the machine-learning pipeline are necessary for KTSP and which are not**.

In particular, I investigated:

- whether PCA is appropriate or necessary for KTSP;
- how gene-pair scores are calculated from expression data;
- how selecting the top K genes affects computation;
- how KTSP compares with conventional machine-learning models;
- how KTSP can be extended to multi-class classification through a hierarchical structure;
- how the research code can be reorganized into a reusable pipeline capable of handling larger datasets.

2. Investigation of PCA and Dimensionality Reduction

One of the first questions I investigated was whether **PCA should be applied before KTSP**.

Cancer gene-expression datasets can contain thousands of genes but relatively few patient samples. PCA can substantially reduce the dimensionality of the input matrix.

For example:

$$X \in \mathbb{R}^{n \times p}$$

where:

- n = number of patients;
- p = number of genes.

PCA transforms this into:

$$X' \in \mathbb{R}^{n \times k}$$

where $k \ll p$.

I incorporated PCA into the pipeline and compared the resulting computational behavior and classification performance with the original gene-expression representation.

Important finding

The investigation showed that **PCA is useful for some conventional classifiers, but it is not intrinsically required by KTSP.**

This distinction is important.

KTSP does not classify patients using PCA components. Instead, it directly compares the relative expression levels of genes:

$$x_i > x_j$$

For a gene pair (i,j) , the classifier determines whether the expression of gene i is greater than gene j in a particular patient.

Therefore, PCA changes the representation from **biologically interpretable genes** to **linear combinations of genes**.

For KTSP, this can reduce interpretability because:

KTSP asks which genes have a characteristic ordering relationship,
whereas PCA replaces genes with mathematical components.

This led to an important architectural distinction in the pipeline:

PCA → useful preprocessing option

KTSP → can operate directly on gene-expression data

This was one of the main methodological findings of the project.

3. Gene-Pair Scoring and Top-K Feature Selection

A central part of the project was understanding how KTSP determines which genes are informative.

Instead of asking:

"How strongly is gene A expressed?"

KTSP asks:

"Is gene A expressed more strongly than gene B?"

For two genes i and j , the ordering can be represented as:

$S_{ij}(x) = \begin{cases} 1 & x_i > x_j \\ 0 & x_i \leq x_j \end{cases}$

The usefulness of a gene pair is then measured by how consistently this ordering distinguishes different biological classes.

For example, suppose a pair behaves differently between two cancer-response groups:

Patient	Gene A	Gene B	A > B
Responder	8.2	5.1	1
Responder	7.9	4.8	1
Non-responder	4.2	7.1	0
Non-responder	4.5	6.8	0

The pair (Gene A, Gene B) therefore becomes a potentially useful classifier feature.

Top-K selection

The complete gene-pair search can become expensive when the number of genes is large.

Instead of evaluating every possible gene equally, the pipeline can first identify a subset of informative genes:

$p \rightarrow K, K \ll p$

and then evaluate gene pairs within this subset.

For example:

16,000 genes→300 top genes→gene pairs→top K pairs

This provides a major reduction in the search space.

My implementation therefore exposed parameters such as:

- top_genes
- top_pairs

rather than hard-coding a single configuration.

This makes it possible to investigate the trade-off between:

computational cost ↔ number of candidate genes ↔ predictive performance.

4.Comparison with Conventional Machine Learning

To determine whether KTSP provides a meaningful alternative to conventional classifiers, I integrated several models into the same pipeline:

- Linear SVM
- RBF SVM
- Decision Tree
- Naive Bayes
- kNN
- KTSP
- Shrinkage-based classifier

The models share the same dataset split and evaluation framework where appropriate.

This provides a controlled way to compare:

Performance=f(model,preprocessing,feature selection)

rather than evaluating KTSP in isolation.

The pipeline records metrics such as:

- Accuracy
- ROC-AUC
- Confusion Matrix

- Predictions
- Model artifacts

This makes the project not simply a KTSP implementation, but an **experimental framework for studying KTSP against alternative classifiers**.

Main Contributions

The main contributions of my work can therefore be summarized as follows:

1. Methodological contribution

I investigated the role of PCA in a K-TSP pipeline and demonstrated the conceptual distinction between PCA-based dimensionality reduction and the direct gene-order comparisons used by KTSP.

2. Feature-selection contribution

I implemented configurable top-gene and top-pair selection and investigated how reducing the candidate gene space affects the computational cost of KTSP.

3. Algorithmic contribution

I developed a hierarchical structure that extends binary K-TSP classification to multi-class datasets.

4. Experimental contribution

I integrated KTSP with conventional classifiers under a common evaluation framework, enabling controlled comparison of model performance.

5. Engineering contribution

I reorganized the research implementation into reusable dataset, preprocessing, training, evaluation, and model-storage components.

6. Scalability contribution

I explored a Ray-based execution architecture that separates the **scientific pipeline from the computational infrastructure**, allowing the same KTSP implementation to run locally or on a larger CPU/GPU server.

Conclusion

The primary contribution of this project is not simply implementing another classifier.

It is the investigation of **how a biologically interpretable K-TSP model can be transformed from a research script into a reproducible, extensible, and potentially scalable cancer-data analysis pipeline.**

The work established an important distinction:

PCA can accelerate and simplify some machine-learning workflows, but KTSP itself does not require PCA because its fundamental operation is based on relative gene-expression ordering.

The resulting pipeline provides a foundation for further experiments involving larger cancer datasets, additional feature-selection strategies, hierarchical classification, and distributed execution.

Current status: Research / experimental pipeline.

Intended use: Computational research and model development, not clinical diagnosis or treatment decisions.